

UNIVERSIDADE TUIUTI DO PARANÁ

**CAIO FEDERICO ESQUIVEL LOVERA ARZE
LUCAS TOTEROL RODRIGUES**

RELATÓRIO DE ESTUDO DIRIGIDO: PLAYER DE MÚSICA EM C

**CURITIBA
2026**

**CAIO FEDERICO ESQUIVEL LOVERA ARZE
LUCAS TOTEROL RODRIGUES**

RELATÓRIO DE ESTUDO DIRIGIDO: PLAYER DE MÚSICA EM C

Estudo dirigido apresentado ao curso de Análise e Desenvolvimento de Sistemas, da Universidade Tuiuti do Paraná, como requisito avaliativo do 2º bimestre da disciplina de Estrutura de Dados.

Orientador: Prof. Luiz Adolpho Baroni

**CURITIBA
2026**

RESUMO

Este trabalho descreve um player de música interativo escrito em linguagem C. A proposta foi colocar em prática as estruturas de dados dinâmicas vistas no bimestre: lista duplamente encadeada, pilha e fila, todas com alocação dinâmica de memória. A biblioteca de músicas é armazenada em uma lista duplamente encadeada. O histórico de reprodução é uma pilha encadeada, e a fila de reprodução, uma fila encadeada. As playlists também são listas duplamente encadeadas, mas cada uma mantém internamente uma lista de invólucros, que apontam para as músicas da biblioteca sem copiá-las. Para a reprodução dos arquivos, integrou-se a biblioteca miniaudio, e a leitura de teclado sem bloqueio permite ao usuário controlar a reprodução em tempo real. O código foi separado em módulos, com um diretório por responsabilidade. Durante o desenvolvimento surgiram dificuldades relevantes: impedir a remoção de músicas em uso, evitar dependência circular entre os módulos, integrar o áudio em tempo real e viabilizar a entrada de teclado não bloqueante nos dois sistemas operacionais. Ao final, o sistema mostrou-se funcional, com menu interativo, reprodução real dos arquivos e navegação completa entre músicas, playlists e histórico.

Palavras-chave: Estruturas de dados; Lista duplamente encadeada; Pilha; Fila; Linguagem C; Player de música.

LISTA DE SIGLAS E ACRÔNIMOS

API	Application Programming Interface (Interface de Programação de Aplicações)
CLI	Command Line Interface (Interface de Linha de Comando)
ED	Estudo Dirigido
FIFO	First In, First Out (Primeiro a Entrar, Primeiro a Sair)
IA	Inteligência Artificial
LIFO	Last In, First Out (Último a Entrar, Primeiro a Sair)

SUMÁRIO

1	INTRODUÇÃO	6
1.1	OBJETIVOS	6
1.2	DESCRIÇÃO DO PROBLEMA	6
2	FUNDAMENTAÇÃO TEÓRICA	8
2.1	ESTRUTURA DE DADOS: LISTA DUPLAMENTE ENCADEADA	8
2.2	ESTRUTURA DE DADOS: PILHA ENCADEADA	8
2.3	ESTRUTURA DE DADOS: FILA ENCADEADA	8
2.4	GERENCIAMENTO DE MEMÓRIA E POLÍTICA DE REMOÇÃO	8
2.5	A ESCOLHA DAS ESTRUTURAS	9
3	PLANEJAMENTO INICIAL	10
3.1	INTERPRETAÇÃO DO PROBLEMA	10
3.2	ESTRUTURAS QUE SERÃO UTILIZADAS	10
4	IMPLEMENTAÇÃO	11
4.1	INICIALIZAÇÃO DO REPOSITÓRIO DO GITHUB	11
4.2	ESTRUTURA DE ARQUIVOS E MAKEFILE	11
4.3	IMPLEMENTAÇÃO DA BIBLIOTECA (LISTA DUPLAMENTE ENCADEADA)	12
4.4	IMPLEMENTAÇÃO DA PILHA ENCADEADA	12
4.5	IMPLEMENTAÇÃO DA FILA ENCADEADA	12
4.6	IMPLEMENTAÇÃO DAS PLAYLISTS	12
4.7	INTEGRAÇÃO DE ÁUDIO REAL	12
4.8	CONTROLE DE TECLADO NÃO BLOQUEANTE	13
4.9	LOOP DE REPRODUÇÃO AO VIVO	13
4.10	TESTES E VALIDAÇÕES	13
4.11	DIFICULDADES E APRENDIZADOS	14
4.11.1	Gerenciamento de Propriedade de Memória	14
4.11.2	Dependência Circular entre Módulos	14
4.11.3	Wrapper de EntradaPlaylist	14
4.11.4	Integração da Biblioteca miniaudio	14
4.11.5	Teclado Não Bloqueante Multiplataforma	15
4.11.6	Restauração Segura do Terminal	15
4.11.7	Pausa Portável sem usleep	15
4.11.8	Tratamento de Buffer de Entrada nos Menus	15
4.12	DECISÕES FEITAS DURANTE O DESENVOLVIMENTO	15
4.13	USO DE IA NO PROJETO	16
5	TECNOLOGIAS UTILIZADAS	17
6	DISTRIBUIÇÃO DE TAREFAS	18
7	CONSIDERAÇÕES FINAIS	19

REFERÊNCIAS	20
--------------------	-----------

1 INTRODUÇÃO

Neste estudo dirigido foi desenvolvido um player de música interativo em linguagem C. A intenção era aplicar na prática as estruturas de dados dinâmicas trabalhadas no segundo bimestre. O trabalho anterior usava vetores de tamanho fixo; aqui adotou-se a alocação dinâmica de memória, com listas encadeadas, pilhas e filas construídas a partir de ponteiros.

O sistema permite cadastrar músicas em uma biblioteca, organizá-las em playlists, enfileirar faixas para tocar em sequência e retornar pelo histórico de reprodução. A reprodução é real, realizada pela biblioteca miniaudio, e o controle durante a execução ocorre diretamente pelo teclado, sem a necessidade de pressionar ENTER a cada comando.

O foco principal era demonstrar como combinar essas estruturas para resolver um problema concreto. Além disso, evidenciou-se o ganho do encadeamento em memória frente aos vetores estáticos: não há limite fixo de capacidade e os nodos são alocados conforme a necessidade.

1.1 OBJETIVOS

O objetivo geral foi construir um sistema em C que simulasse um player de música completo, com gerenciamento de biblioteca, playlists, fila de reprodução e histórico, sempre apoiado em estruturas de dados dinâmicas. Para chegar nesse resultado, definiram-se os seguintes objetivos específicos:

- Implementar uma lista duplamente encadeada para a biblioteca de músicas e para as playlists;
- Usar uma pilha encadeada como histórico de reprodução;
- Usar uma fila encadeada como fila de reprodução;
- Integrar a reprodução real de áudio por meio da biblioteca miniaudio;
- Implementar o controle de teclado sem bloqueio em mais de um sistema operacional;
- Organizar o código de forma modular, com um diretório por responsabilidade.

1.2 DESCRIÇÃO DO PROBLEMA

O sistema simula um player completo operado pelo terminal. No cadastro, o usuário informa título, artista e caminho do arquivo, enquanto a duração é detectada

automaticamente pelo próprio motor de áudio. As músicas ficam na biblioteca, uma lista duplamente encadeada que pode ser percorrida nos dois sentidos.

Durante a reprodução, é possível avançar ou retroceder faixas, pular para a próxima música da fila, ativar o modo aleatório (shuffle) ou pausar e retomar. Cada música que toca entra no histórico de forma automática. Também é possível criar playlists, adicionar músicas a elas e tocar a partir delas.

Há uma regra que precisa ser respeitada: uma música não pode ser removida da biblioteca enquanto ainda estiver referenciada no histórico, na fila ou em alguma playlist. Sem essa proteção, os ponteiros armazenados nessas estruturas passariam a referenciar memória inválida.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo reúne os conceitos que serviram de base para o sistema, com apoio nas referências utilizadas (BHARGAVA, 2017; BARONI, 2026).

2.1 ESTRUTURA DE DADOS: LISTA DUPLAMENTE ENCADEADA

Na lista duplamente encadeada, cada nodo guarda dois ponteiros: um para o nodo seguinte (`prox`) e outro para o anterior (`ant`). É essa bidirecionalidade que permite percorrer a lista nas duas direções sem ter de voltar sempre ao início.

Como cada nodo é alocado dinamicamente, não existe tamanho máximo definido em tempo de compilação. Tendo um ponteiro para o nodo alvo, inserir ou remover no meio da lista custa $O(1)$, bastando ajustar os ponteiros dos vizinhos. No projeto, essa estrutura aparece na biblioteca principal de músicas e também na lista interna de cada playlist.

2.2 ESTRUTURA DE DADOS: PILHA ENCADEADA

A pilha encadeada segue a política LIFO (Last In, First Out) sobre uma lista simplesmente encadeada, em que o topo é sempre o primeiro nodo. Empilhar (`push`) e desempilhar (`pop`) custam $O(1)$, pois atuam apenas no topo, e a estrutura cresce conforme a demanda, sem limite fixo.

Aqui a pilha guarda ponteiros para os `NodoMusica` da biblioteca e funciona como histórico de reprodução. Guardar ponteiros em vez de cópias evita duplicar dados e mantém o histórico sempre apontando para os dados originais da biblioteca.

2.3 ESTRUTURA DE DADOS: FILA ENCADEADA

Já a fila encadeada segue a política FIFO (First In, First Out), também sobre uma lista simplesmente encadeada, mas com ponteiros para o início e para o fim. O enqueue insere no fim e o dequeue remove do início, os dois em $O(1)$. Assim como a pilha, ela não tem limite fixo de capacidade.

No player, a fila de reprodução guarda ponteiros para `NodoMusica` e define a ordem das próximas faixas. Enquanto houver algo nela, a fila tem prioridade sobre a sequência da biblioteca ou da playlist.

2.4 GERENCIAMENTO DE MEMÓRIA E POLÍTICA DE REMOÇÃO

O ponto central do projeto é a diferença entre proprietário e referenciador. A dona dos `NodoMusica` é a biblioteca, e só ela aloca e libera esses nodos. Pilha, fila e playlists guardam apenas ponteiros para eles. Daí a regra de não remover uma

música da biblioteca enquanto alguma estrutura ainda a referencia: liberar o nodo deixaria ponteiros pendentes apontando para memória já liberada, o que resulta em comportamento indefinido.

2.5 A ESCOLHA DAS ESTRUTURAS

A biblioteca usa lista duplamente encadeada porque a navegação nos dois sentidos é essencial, já que avançar e voltar na ordem das músicas são operações igualmente comuns. O histórico pediu naturalmente uma pilha, pois a última música tocada é, em geral, a primeira a ser consultada. Por sua vez, a fila descreve bem uma lista de reprodução agendada, em que a primeira faixa enfileirada é a primeira a tocar.

3 PLANEJAMENTO INICIAL

3.1 INTERPRETAÇÃO DO PROBLEMA

A partir do enunciado, levantaram-se as principais entidades do sistema: `NodoMusica` (título, artista, caminho do arquivo e duração), `ListaDupla` (a biblioteca), `Pilha` (o histórico), `Fila` (a fila de reprodução), `NodoPlaylist` (nome e lista interna de músicas) e `EstadoPlayer` (o estado atual da reprodução).

O fluxo básico ficou assim:

1. Cadastro de músicas pelo usuário, com a duração detectada automaticamente pelo áudio;
2. Reprodução de uma música, que envolve carregar o arquivo no engine, empilhar no histórico e atualizar o estado;
3. Loop de reprodução ao vivo, atualizando a barra de status e lendo as teclas sem bloquear;
4. No fim de cada música, olhar a fila, a playlist ou a biblioteca para decidir a próxima faixa;
5. Gerenciamento de playlists, com criação, adição e remoção de músicas e reprodução a partir delas;
6. Remoção segura de músicas, checando o uso em pilha, fila e playlists antes de liberar o nodo.

3.2 ESTRUTURAS QUE SERÃO UTILIZADAS

Depois dessa análise, definiram-se as escolhas:

- **Lista Duplamente Encadeada:** para a biblioteca e para a lista interna de cada playlist, por causa da navegação nos dois sentidos.
- **Pilha Encadeada:** para o histórico, em que o acesso mais recente é o mais relevante.
- **Fila Encadeada:** para a fila de reprodução, respeitando a ordem FIFO.
- **Lista de Playlists:** também uma lista duplamente encadeada de `NodoPlaylist`, com cada nodo carregando uma lista interna de `EntradaPlaylist` (os invólucros com ponteiro para `NodoMusica`).
- **Structs em tipos.h:** para centralizar as definições de tipos e cortar a dependência circular entre os módulos.

4 IMPLEMENTAÇÃO

O desenvolvimento foi colaborativo do início ao fim. Os dois integrantes discutiram cada decisão de projeto e escreveram o código juntos, usando pair programming.

4.1 INICIALIZAÇÃO DO REPOSITÓRIO DO GITHUB

Para controle de versão e colaboração, criou-se um repositório privado no GitHub. A cada novo módulo, eram realizados commits com mensagens descritivas. Isso manteve o histórico do desenvolvimento rastreável e permitiu reverter alterações quando necessário.

4.2 ESTRUTURA DE ARQUIVOS E MAKEFILE

A organização é modular, com um diretório para cada componente:

- `tipos.h`: definições centrais de todos os tipos (`NodoMusica`, `Pilha`, `Fila`, `ListaPlaylists`, `EstadoPlayer`);
- `biblioteca_func/`: lista duplamente encadeada da biblioteca;
- `pilha_func/`: pilha encadeada (histórico);
- `fila_func/`: fila encadeada (fila de reprodução);
- `playlist_func/`: lista de playlists e suas operações;
- `audio_func/`: wrapper sobre a biblioteca miniaudio;
- `input_func/`: teclado não bloqueante (Windows e Linux);
- `player_func/`: loop de reprodução ao vivo;
- `menu_func/`: todos os menus de texto;
- `main.c`: ponto de entrada.

O Makefile detecta automaticamente a plataforma (Linux ou Windows via MinGW) e ajusta o executável gerado (`player` ou `player.exe`), as bibliotecas de link (`-lpthread -lm -ldl` no Linux e `-lole32 -lwinmm` no Windows) e o comando de limpeza. As flags `-Wall -Wextra -std=c99 -g` mantêm o rigor na compilação. Criou-se ainda uma regra extra, `make windows`, para cross-compilar do Linux para Windows com o MinGW-w64.

4.3 IMPLEMENTAÇÃO DA BIBLIOTECA (LISTA DUPLAMENTE ENCADEADA)

A biblioteca é uma `ListaDupla` com ponteiros `inicio` e `fim` e um contador `tam`. Cada `NodoMusica` é alocado dinamicamente e entra no fim da lista. A busca por título e por artista usa correspondência parcial e ignora maiúsculas e minúsculas, passando as strings para minúsculo em buffers temporários antes de chamar o `strstr`. A remoção exige mais cuidado: antes de liberar a memória, verifica, por meio de um callback, se o nodo está sendo usado em outra estrutura.

4.4 IMPLEMENTAÇÃO DA PILHA ENCADEADA

A pilha é uma lista simplesmente encadeada com um ponteiro `topo`. Cada `NodoPilha` guarda apenas um ponteiro para `NodoMusica`, nunca uma cópia. O `push` aloca um `NodoPilha` novo e o encadeia no topo; o `pop` remove esse `NodoPilha` e o libera, mas nunca acessa o `NodoMusica` apontado. A `pilha_liberar` aplica o mesmo princípio em escala: libera apenas os `NodoPilha` e mantém os `NodoMusica` intactos.

4.5 IMPLEMENTAÇÃO DA FILA ENCADEADA

A fila tem ponteiros `inicio` e `fim` e, como a pilha, guarda ponteiros para `NodoMusica` sem duplicar dados. O `enqueue` aloca um `NodoFila` e o encadeia no fim; o `dequeue` remove o primeiro e devolve o ponteiro da música. Ao avançar para a próxima faixa, é a fila que tem prioridade dentro do loop do player.

4.6 IMPLEMENTAÇÃO DAS PLAYLISTS

As playlists ficam em uma `ListaPlaylists`, que é uma lista duplamente encadeada de `NodoPlaylist`. Por dentro, cada playlist tem outra lista duplamente encadeada, dessa vez de `EntradaPlaylist`: invólucros (wrappers) com um ponteiro para `NodoMusica` e `prox/ant` próprios. Essa separação não é arbitrária. Os ponteiros `prox` e `ant` do `NodoMusica` já pertencem à cadeia da biblioteca; reaproveitá-los nas listas internas das playlists corromperia a biblioteca.

4.7 INTEGRAÇÃO DE ÁUDIO REAL

A reprodução é construída sobre a biblioteca miniaudio (REID, 2024) (o arquivo único `miniaudio.h`), um motor de áudio multiplataforma de código aberto escrito em C. Criou-se o módulo `audio_func/` para encapsular toda a API do miniaudio e expor apenas funções de alto nível: `audio_init`, `audio_load`, `audio_play`, `audio_pause`, `audio_resume`, `audio_seek_relative` e `audio_stop`. Nenhum

outro módulo inclui o `miniaudio.h` diretamente, de modo que toda a complexidade da biblioteca fica concentrada em um único local.

4.8 CONTROLE DE TECLADO NÃO BLOQUEANTE

O módulo `input_func/` lê o teclado sem bloqueio nos dois sistemas. No Linux, o terminal é colocado em modo raw com `tcsetattr`, desligando o modo canônico e o eco, e o `select` verifica se há bytes disponíveis antes de chamar o `read` (THE LINUX MAN-PAGES PROJECT, 2026). No Windows, utilizam-se `_kbhit` e `_getch` da `conio.h`. A `input_modo_normal` é registrada com `atexit`, o que restaura o terminal mesmo em uma saída abrupta.

4.9 LOOP DE REPRODUÇÃO AO VIVO

O `player_loop` reescreve a barra de status do terminal a cada 200 ms, cinco vezes por segundo, sobrescrevendo a linha atual com `\r` e chamando `fflush(stdout)`. No mesmo intervalo, ele verifica duas condições: se a música terminou (por meio de `audio_is_at_end`), para avançar automaticamente, e se há tecla disponível, para tratar os comandos. A escolha da próxima música obedece a uma ordem de prioridade: primeiro a fila de reprodução, depois a próxima da playlist atual (ou uma aleatória, no shuffle) e, por último, a próxima da biblioteca (de novo aleatória, se o shuffle estiver ligado).

4.10 TESTES E VALIDAÇÕES

Os testes foram manuais e cobriram:

- Inserção e remoção na lista duplamente encadeada, conferindo `prox` e `ant` nos extremos e no meio;
- Empilhamento e desempilhamento, confirmando que só os `NodoPilha` eram liberados e que os `NodoMusica` seguiam intactos;
- Enfileiramento e desenfileiramento, checando a ordem FIFO;
- Tentativa de remover músicas que estavam em uso na pilha, na fila e em playlists, para ver se as mensagens de bloqueio apareciam;
- Reprodução de arquivos `.mp3` e `.flac` reais, verificando a detecção de duração e o avanço e retrocesso de posição;
- Restauração do terminal depois de sair com `q` ou de uma interrupção abrupta.

4.11 DIFICULDADES E APRENDIZADOS

Algumas etapas exigiram mais esforço e pesquisa adicional. As principais estão descritas a seguir.

4.11.1 Gerenciamento de Propriedade de Memória

Acompanhar qual estrutura é proprietária de um nodo (a biblioteca) e qual apenas o referencia (pilha, fila e playlists) exigiu atenção constante. Liberar um `NodoMusica` com ponteiros ainda apontando para ele daria comportamento indefinido. O problema foi resolvido com a verificação de uso antes de qualquer remoção da biblioteca, somada à garantia de que `pilha_liberar` e `fila_liberar` nunca acessam os `NodoMusica`.

4.11.2 Dependência Circular entre Módulos

A `biblioteca_remove` precisa saber se uma música está em alguma playlist, só que o módulo `biblioteca_func` não pode incluir o `playlist_func` (que inclui o `tipos.h` e fecharia um ciclo). A saída foi um callback: a `biblioteca_remove` recebe um ponteiro de função `em_alguna_playlist` e um `void*` genérico para a lista de playlists. Quem chama, no caso o menu, passa `playlist_em_alguna` e `&playlists`, e os módulos seguem desacoplados.

4.11.3 Wrapper de EntradaPlaylist

Conforme já mencionado, os `prox` e `ant` do `NodoMusica` pertencem à cadeia da biblioteca. Se uma playlist utilizasse esses mesmos ponteiros para encadear as músicas, as duas listas entrariam em conflito. Por isso o tipo `EntradaPlaylist`, um invólucro com `prox` e `ant` próprios e um ponteiro para o `NodoMusica` certo. Assim a mesma música pode estar em várias playlists ao mesmo tempo, sem conflito.

4.11.4 Integração da Biblioteca miniaudio

A miniaudio é header-only, portanto a macro `MINIAUDIO_IMPLEMENTATION` deve ser definida em exatamente um arquivo `.c` antes do header (REID, 2024). Defini-la em mais de um arquivo geraria símbolo duplicado no linker. Para evitar isso, manteve-se a macro apenas no `audio_func/audio.c` e adicionou-se uma regra própria no Makefile para compilar esse arquivo sem as flags de aviso extras, que geram um volume excessivo de alertas quando aplicadas a código de terceiros.

4.11.5 Teclado Não Bloqueante Multiplataforma

No Linux, a entrada do terminal é canônica por padrão: os caracteres ficam no buffer até o ENTER. Para o loop em tempo real, era necessário ler tecla por tecla, sem bloquear. Implementou-se o `input_modo_raw` com `tcgetattr/tcsetattr` para desligar o modo canônico e o eco, e utilizou-se `select` com timeout zero para verificar se há tecla disponível sem permanecer em espera (THE LINUX MAN-PAGES PROJECT, 2026). No Windows, `_kbhit/_getch` fornecem esse comportamento nativamente.

4.11.6 Restauração Segura do Terminal

Caso o programa fosse encerrado abruptamente (por um Ctrl+C, por exemplo) com o terminal em modo raw, o usuário ficaria com o terminal inutilizável. Registrou-se a `input_modo_normal` com `atexit` para garantir a restauração em qualquer caminho de saída.

4.11.7 Pausa Portável sem usleep

Esperar 200 ms entre as iterações do loop sem `usleep` (que pede `_BSD_SOURCE` ou `_XOPEN_SOURCE >= 500` e não existe no Windows) levou a uma função auxiliar, a `dormir_ms`, baseada no `nanosleep` do POSIX.1b (THE LINUX MAN-PAGES PROJECT, 2026). No Windows, o equivalente é o `Sleep` da `windows.h`, escolhido pela macro `_WIN32`.

4.11.8 Tratamento de Buffer de Entrada nos Menus

O mesmo problema do trabalho anterior reapareceu: o `scanf` deixa o `\n` no buffer, prejudicando a leitura seguinte. A solução foi consumir os caracteres restantes com `getchar()` em um laço até encontrar `\n` ou EOF, sem recorrer ao `fflush(stdin)`, cujo comportamento é indefinido fora do Windows (CPPREFERENCE.COM, 2026).

4.12 DECISÕES FEITAS DURANTE O DESENVOLVIMENTO

As principais decisões de arquitetura e design ao longo do projeto foram estas:

1. **Tipos centralizados em tipos.h:** juntar todos os `typedef` num único arquivo, incluído por todos os módulos, evitou espalhar forward declarations e simplificou as dependências.
2. **Pilha e fila guardam ponteiros, não cópias:** duplicar os `NodoMusica` em cada estrutura desperdiçaria memória e ainda exigiria sincronizar tudo a cada alteração.

Com ponteiros, todas as estruturas compartilham a mesma fonte de verdade, a biblioteca.

3. **Wrapper EntradaPlaylist:** como os ponteiros de encadeamento do `NodoMusica` já estavam em uso pela biblioteca, o invólucro separado foi o que deixou a mesma música aparecer em várias playlists sem corromper nenhuma lista.
4. **Callback na biblioteca_remove:** em vez de criar dependência direta da `biblioteca_func` sobre a `playlist_func`, passou-se a função de verificação como parâmetro, mantendo o acoplamento no mínimo.
5. **Encapsulamento total da miniaudio:** só o `audio_func` inclui o `miniaudio.h`. Isso isola a API de terceiros e facilita uma eventual substituição do motor de áudio.
6. **atexit para restaurar o terminal:** garantir o terminal restaurado em qualquer saída foi decisão de robustez. Sem ela, uma saída por sinal ou por `exit()` direto deixaria o terminal inutilizável.
7. **Makefile multiplataforma:** suportar Linux e Windows no mesmo Makefile, com a regra `make windows` para cross-compilar com MinGW-w64, simplificou a distribuição do executável.

4.13 USO DE IA NO PROJETO

Utilizaram-se assistentes de IA (ChatGPT e Claude) como apoio em alguns pontos:

- Dúvidas sobre a API do `tcsetattr` e do `select` para o modo raw do terminal no Linux;
- As restrições da macro `MINIAUDIO_IMPLEMENTATION` e como fugir de símbolos duplicados no linker;
- A ideia do callback para quebrar a dependência circular entre biblioteca e playlist;
- A escrita do Makefile multiplataforma, com detecção automática do sistema operacional.

Todo o código gerado ou sugerido por IA passou por revisão, teste e integração pela equipe, de modo que cada decisão tomada foi compreendida. O uso dessas ferramentas segue as diretrizes do professor, que incentiva o aprendizado crítico e a validação das informações.

5 TECNOLOGIAS UTILIZADAS

As tecnologias e ferramentas usadas no estudo dirigido foram:

- **Linguagem C (padrão C99)**: para todo o sistema.
- **GCC (GNU Compiler Collection)**: compilador no Linux e no Windows (via MinGW-w64).
- **Make / mingw32-make**: automação da compilação pelo Makefile.
- **miniaudio**: biblioteca header-only de código aberto para áudio em C, com suporte a MP3, WAV, FLAC e outros formatos.
- **Git e GitHub**: controle de versão e hospedagem do repositório.
- **Visual Studio Code**: editor de código com extensões para C/C++.
- **Valgrind (Linux)**: detecção de vazamentos de memória, utilizado para confirmar que só os nodos de encadeamento eram liberados, e não os `NodoMusica`.

6 DISTRIBUIÇÃO DE TAREFAS

Como o enunciado pede, segue a divisão de responsabilidades entre os integrantes ao longo do desenvolvimento.

LUCAS TOTEROL RODRIGUES

- Módulo de fila encadeada (`fila_func/`), com todas as operações (enqueue, dequeue, peek, contem, listar) e a liberação seletiva dos nodos.
- Módulo de playlists (`playlist_func/`), com criação e remoção de playlists, adição e remoção de músicas via `EntradaPlaylist` e a navegação sequencial.
- Integração do avanço e retrocesso de músicas dentro das playlists no loop do player.

CAIO FEDERICO ESQUIVEL LOVERA ARZE

- Módulo de pilha encadeada (`pilha_func/`), com todas as operações (push, pop, peek, contem, listar) e a liberação seletiva dos nodos.
- Módulo de áudio (`audio_func/`), integrando a miniaudio e expondo a API de alto nível de reprodução, pausa, seek e detecção de fim de faixa.
- Módulo de entrada não bloqueante (`input_func/`), com suporte a Linux (`tcsetattr/select`) e Windows (`_kbhit/_getch`).

TRABALHO CONJUNTO

- Biblioteca de músicas (`biblioteca_func/`), com cadastro, remoção com verificação de uso, busca e listagem.
- Loop de reprodução ao vivo (`player_func/`), com a barra de status, o tratamento de teclas e a prioridade da próxima música.
- Menus de texto (`menu_func/`), juntando todas as estruturas numa interface de linha de comando coerente.
- Arquivo central `tipos.h`, com as structs e typedefs compartilhados.
- Makefile multiplataforma, incluindo a regra de cross-compilação para Windows.
- Testes manuais com Valgrind e validação do sistema.
- Elaboração deste relatório técnico.

7 CONSIDERAÇÕES FINAIS

O estudo dirigido permitiu aplicar na prática as estruturas de dados dinâmicas (lista duplamente encadeada, pilha encadeada e fila encadeada) em um player de música plenamente funcional em C. A reprodução real de áudio, o teclado sem bloqueio e as playlists tornaram o projeto consideravelmente mais complexo que o anterior, o que exigiu mais cuidado com a memória e com o desenho da arquitetura.

O conceito que mais pesou foi a separação entre proprietário e referenciador de nodos. Entender que só a biblioteca aloca e libera os `NodoMusica`, enquanto pilha, fila e playlists apenas apontam para eles, evitou erros difíceis de depurar, como ponteiro inválido e dupla liberação. Esse padrão de propriedade clara é básico no gerenciamento manual de memória em C.

Quebrar a dependência circular com o callback e criar invólucros como a `EntradaPlaylist` para reutilizar estruturas em contextos diferentes foram aprendizados de design de software que vão além da simples codificação de uma estrutura de dados.

Por fim, o projeto mostrou que escolher bem as estruturas e deixar claras as responsabilidades de cada módulo conta tanto quanto a correção do código. Fica como base para sistemas mais complexos, que dependem de um gerenciamento eficiente e seguro dos dados em memória.

REFERÊNCIAS

- BARONI, L. A. **Estruturas de Dados: Aulas da Disciplina**. 2026. Material didático (slides) disponibilizado na disciplina. Universidade Tuiuti do Paraná, Curitiba.
- BHARGAVA, A. Y. **Entendendo Algoritmos: Um Guia Ilustrado para Programadores e Outros Curiosos**. São Paulo: Novatec, 2017.
- CPPREFERENCE.COM. **fflush – C reference**. 2026. Documentação da biblioteca padrão da linguagem C. Acesso em: maio 2026. Disponível em: <https://en.cppreference.com/w/c/io/fflush>.
- REID, D. **miniaudio: A single file audio playback and capture library written in C**. 2024. Documentação oficial. Acesso em: maio 2026. Disponível em: <https://miniaud.io/docs/>.
- THE LINUX MAN-PAGES PROJECT. **Linux man-pages: tcsetattr(3), select(2) e nanosleep(2)**. 2026. Documentação do kernel Linux e da interface POSIX, baseada em IEEE Std 1003.1. Acesso em: maio 2026. Disponível em: <https://man7.org/linux/man-pages/>.